

KARADENİZ TEKNİK ÜNİVERSİTESİ — BİLGİSAYAR BİLİMLERİ

Web Programlama

Veritabanı: SQLAlchemy ve Kalıcılık

Hafta 8

Veritabanı: SQLAlchemy ve Kalıcılık

Doç. Dr. Tolga Berber

KTÜ Fen Fakültesi, Bilgisayar Bilimleri Bölümü · https://avesis.ktu.edu.tr/tberber

Öğrenme fazları: 1 • Ön-öğretim | 2 • İçerik | 3 • Katılım | 4 • Değerlendirme | 5 • İzleme

1 • Ön-öğretim Neden Önemli?

Hf.6-7'de gerçek bir sunucu ve gerçek bir ön yüz kurdunuz — ama sunucunun hafızası HÂLÂ geçiciydi: her `uvicorn --reload` yeniden başladığında girdiğiniz tüm etkinlik/koltuk/bilet verisi sıfırlanıyordu.

- Dönem projeniz için "her ders başında verileri yeniden girme" kabul edilemez — bugün bu veriyi bir dosyaya yazmanın yolunu öğreniyorsunuz
- Hf.6'nın "aynı koltuğu iki kullanıcı aynı anda alamaz" kuralı (`409`) şimdiye kadar yalnızca bir şans eseriydi — tek Python süreci istekleri pratikte sırayla işliyordu
- Bugün bu şans, gerçek bir veritabanı transaction'ına dönüşüyor

SQLAlchemy ile veriniz artık bilet.db dosyasında kalıcı — sunucuyu kapatıp açsanız bile yerinde durur.

Bu Hafta

◎ Öğrenme Hedefleri

Bu hafta sonunda şunları yapabileceksiniz:

1. `DeclarativeBase`/`Mapped`/`mapped_column` ile ORM modelleri tanımlayıp `engine`/`Session` kurarak `bilet.db` dosyasında tablo oluşturmak
2. `ForeignKey` ve `relationship` ile Etkinlik–Koltuk– Bilet ilişkilerini kurup tembel/istekli yükleme ve cascade silmeyi uygulamak
3. `select()`/`limit`/`offset` ile sayfalanmış sorgular yazıp `echo=True` ile üretilen SQL'i okumak
4. bir `Session` içinde `commit`/`rollback` disipliniyle tek-transaction rezervasyon uygulayıp bir `pytest` testi yazmak
5. Etkinlik/Bilet Sisteminin deposunu bellekten SQLite'a taşıyarak kalıcılık ve gerçek eşzamanlı rezervasyon koruması eklemek

BÖLÜM 1

Ünite 1 — ORM Kavramı, Model/Tablo, Oturum

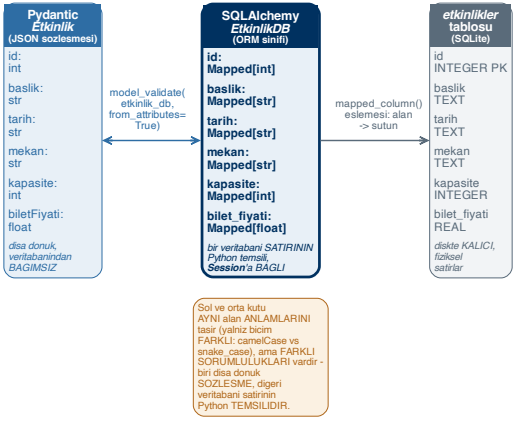
2 • İçerik ORM, Bildirimsel Model, `engine`, `Session`

- YG II'de SQLite'a elle SQL yazıp dönen satırları kendi elinizle bir yapıya çevirdiniz — bu tekrar tekrar yazılan kod, ORM'in otomatikleştirdiği iştir
- SQLAlchemy 2.x bildirimsel stil: `class EtkinlikDB(Base):` içindeki her `Mapped[tip]` + `mapped_column(...)` bir sütun tanımı üretir
- `engine` veritabanı dosyasına bağlanan, uygulama başına genelde tek nesne; `Session` bir "çalışma birimi" — her istek için ayrı açılıp kapatılır
- Kritik ayrım: `EtkinlikDB` (ORM, bir `Session`'a bağlı) ile `Etkinlik` (Pydantic, dışa dönük JSON sözleşmesi) aynı sınıf değildir — `model_validate(from_attributes=True)` ikisini köprüler

Tanım: ORM (nesne-ilişkisel eşleme; *ORM*)

Python sınıflarını veritabanı tablolarına, nesneleri satırlara eşleyen katmandır.

Üç Katman: Pydantic → ORM → SQL Tablosu



Analoji: Tercüman ve Form Şablonu

Analoji

ORM'i bir tercümana benzetin: Python nesneleriniz Python "konuşur", veritabanı SQL "konuşur" — siz Python nesnesiyle konuşursunuz, tercüman arka planda doğru SQL cümlesini kurar ve veritabanının cevabını yine sizin dilinize çevirir.

Analoji

Bildirimsel modeli bir kayıt masasının basılı form şablonuna benzetin: `EtkinlikDB` sınıfının tanımı hangi alanın (ad, tarih, sayı) olacağını önceden belirler; her yeni kayıt bu şablondan bir kopya doldurulmasıdır.

Kod: engine, Taban Sınıf, Oturum Fabrikası — app/veritabani.py

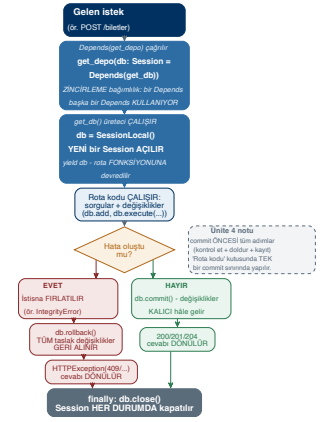
```
# echo=True açıldığında (Ünite 3) her çalıştırılan SQL cümlesi konsola
# basılır; günlük çalışmada gürültü yapmaması için varsayılan False'tur.
engine = create_engine("sqlite:///bilet.db", echo=False)
sqlite_yazma_kilidi_uygula(engine)
```

```
class Base(DeclarativeBase):
    """Tüm ORM modellerinin (EtkinlikDB, KoltukDB, BiletDB) miras aldığı taban sınıf.""
        "
```

```
SessionLocal = sessionmaker(bind=engine)
```

`Base`'i miras alan her sınıf (`EtkinlikDB`, `KoltukDB`, `BiletDB`) otomatik olarak birer tablo tanımı olur.

İstek Yaşam Döngüsü: `get_db` → Oturum → Kapanış



Kod: `get_db` Bağımlılık Üreteci — `app/veritabani.py`

```python
def get_db() -> Generator[Session, None, None]:
    """Her istek için YENİ bir Session açar, istek bitince KAPATIR.

    `yield`'den ÖNCEKİ blok isteği İŞLEYEN koddan ÖNCE, `finally` bloğu
    isteği İŞLEYEN kod bittikten SONRA (hata OLSA bile) çalışır.
    """
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

`yield`'den önceki blok isteği işleyen koddan önce, `finally` bloğu ise sonra (hata olsa bile) çalışır — Hf.6 Ünite 4'ün "Hf.8'de bu yer tutucuyu değiştirecek" notu burada gerçekleşiyor.

Kod: ORM Modeli Tanımı — `app/orm_modeller.py`

```python
class EtkinlikDB(Base):
    """Bir `etkinlikler` satırının Python temsili."""

    __tablename__ = "etkinlikler"

    id: Mapped[int] = mapped_column(primary_key=True)
    baslik: Mapped[str]
    tarih: Mapped[str]
    mekan: Mapped[str]
    kapasite: Mapped[int]
    bilet_fiyati: Mapped[float]

    # Bir etkinlik SİLİNDİĞİNDE bağlı TÜM koltuklar (ve onların cascade'ine
    # bağlı biletleri) OTOMATİK silinir — Ünite 2'nin cascade silme kuralı.
    koltuklar: Mapped[list["KoltukDB"]] = relationship(
        back_populates="etkinlik", cascade="all, delete-orphan"
    )
```

`id: Mapped[int] = mapped_column(primary_key=True)` satırı hem sütun tipini hem prımary key kısıtını aynı anda tanımlar — `koltuklar` alanı bir sonraki ünitenin ilişki konusuna giriştir.

BÖLÜM 2

Ünite 2 — İlişkiler: Etkinlik–Koltuk–Bilet

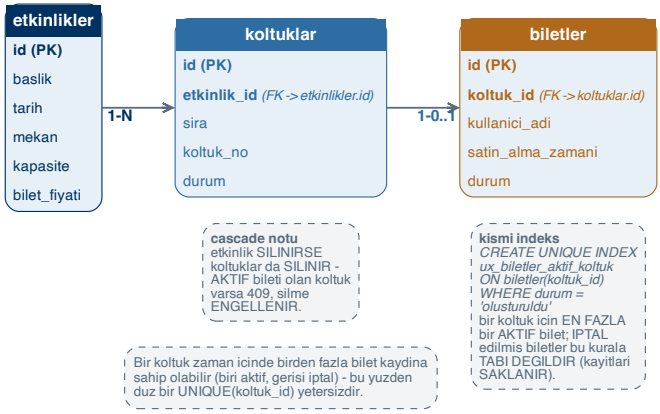
2 • İçerik ForeignKey, relationship, N+1, Cascade Silme

- Bir etkinliğin birden fazla koltuğu olabilir, ama her koltuk yalnızca bir etkinliğe aittir — `ForeignKey("etkinlikler.id")` bu kısıtı veritabanı düzeyinde kurar
- `relationship(back_populates=...)` Python tarafında rahat erişimi (`etkinlik.koltuklar`) sağlar — elle bir `JOIN` yazmanıza gerek kalmaz
- N+1 sorunu: `lazy="select"` (varsayılan) her etkinlik için ayrı bir sorgu tetikler — `selectinload(...)` ile tüm ilişkiler tek bir ek sorguda önceden çekilir
- Cascade silme (bu hafta kesinleşir): satılmış bileti olan bir koltuk varsa etkinlik silme `409` ile reddedilir; yoksa `cascade="all, delete-orphan"` bağlı koltukları otomatik siler

Tanım: İlişki (*relationship*)

ORM'de iki modelin yabancı anahtar üzerinden birbirine bağlı nesneler olarak erişilmesini sağlayan tanımdır.

Er Şeması: 1–N ve 1–0..1 İlişkiler



Analoji: Çapraz Referans ve Sınıf Listesi

Analoji

İlişki/yabancı anahtarı bir rezervasyon defterindeki çapraz referansa benzetin: koltuk satırındaki bir sütun ”hangi etkinliğe ait” diye o etkinliğin sıra numarasını taşır — defterin sayfaları böyle birbirine bağlanır.

Analoji

Tembel/istekli yüklemeyi bir öğretmenin sınıf listesini almasına benzetin: önce listeyi görüp her öğrencinin notunu ayrı ayrı dosyadan çekmek (tembel — her öğrenci için ayrı bir gidiş) ile hepsini tek seferde getiren bir rapor istemek (istekli) arasındaki fark.

Kod: `KoltukDB` — Yabancı Anahtar ve İlişki

```python
class KoltukDB(Base):
    """Bir `koltuklar` satırının Python temsili — HER koltuk bir etkinliğe AİTTİR."""

    __tablename__ = "koltuklar"

    id: Mapped[int] = mapped_column(primary_key=True)
    etkinlik_id: Mapped[int] = mapped_column(ForeignKey("etkinlikler.id"))
    sira: Mapped[int]
    koltuk_no: Mapped[int]
    durum: Mapped[str] = mapped_column(default="bos")

    etkinlik: Mapped["EtkinlikDB"] = relationship(back_populates="koltuklar")
    biletler: Mapped[list["BiletDB"]] = relationship(
       back_populates="koltuk", cascade="all, delete-orphan"
    )
```

`etkinlik: Mapped["EtkinlikDB"]` ile `EtkinlikDB.koltuklar` `back_populates` sayesinde birbirini karşılıklı tanır.

Kod: `BiletDB` — Neden Düz `UNIQUE` Yetersiz

```python
class BiletDB(Base):
    """Bir `biletler` satırının Python temsili.

    İptal edilmiş biletler SİLİNMEZ, `durum='iptal'` olarak SAKLANIR
    (denetim amaçlı) — bu yüzden bir koltuğun zaman içinde BİRDEN FAZLA
    bilet kaydı olabilir (biri aktif, gerisi iptal).
    """

    __tablename__ = "biletler"

    id: Mapped[int] = mapped_column(primary_key=True)
    koltuk_id: Mapped[int] = mapped_column(ForeignKey("koltuklar.id"))
    kullanici_adi: Mapped[str]
    satin_alma_zamani: Mapped[str]
    durum: Mapped[str] = mapped_column(default="olusturuldu")

    koltuk: Mapped["KoltukDB"] = relationship(back_populates="biletler")
```

İptal edilmiş biletler silinmez, `durum='iptal'` olarak saklanır — bir koltuk zaman içinde birden fazla bilet kaydına sahip olabilir; çözümü (kısmi `UNIQUE` indeks) Ünite 4'te.

Kod: Cascade Silme — app/depo.py

```
72    def etkinlik_sil(self, etkinlik_id: int) -> str:
81       etkinlik_db = self.db.get(EtkinlikDB, etkinlik_id)
82       if etkinlik_db is None:
83          return "bulunamadi"
84
85       aktif_bilet_var = any(
86          bilet.durum == "olusturuldu"
87          for koltuk in etkinlik_db.koltuklar
88          for bilet in koltuk.biletler
89       )
90       if aktif_bilet_var:
91          return "aktif_bilet_var"
92
93       self.db.delete(etkinlik_db)
94       self.db.commit()
95       return "silindi"
```

aktif bilet varsa `"aktif_bilet_var"` döner (rota katmanı bunu `409`'a çevirir); yoksa `self.db.delete(...)` `cascade="all, delete-orphan"` sayesinde bağlı koltukları otomatik siler.

Analoji: Cascade Silme — Bir Etkinliğin İptali

Analoji

Cascade silmeyi bir etkinliğin iptaline benzetin: bir konser tamamen iptal edildiğinde o konserin boş koltukları da anlamsız hâle gelir ve birlikte silinir — ama bileti satılmış bir koltuk varsa, önce o biletlerin ne olacağına karar vermeden konseri silemezsiniz.

BÖLÜM 3

Ünite 3 — Sorgular, Sayfalama, Ham SQL

2 • İçerik `select()`, Sayfalama, `echo=True`

- `select(EtkinlikDB).where(...).order_by(...)` — YG II'nin `SELECT ... WHERE ... ORDER BY ...` cümlesinin Python nesnesi olarak ifadesidir
- `.limit(boyut).offset((sayfa - 1) * boyut)` — Hf.6'nın sorgu parametresi fikrinin doğrudan devamı: `?sayfa=2&boyut=10`, dönüş tipi hâlâ bir dizidir
- `func.count` toplam kayıt sayısını hesaplar; bir `X-Toplam-Kayit` başlığı olarak dönülebilir (bu haftanın ana konusu değil)
- `echo=True` Her çalıştırılan SQL'i konsola basar — YG II'de elle yazdığınız SQL ile yan yana koyup karşılaştırın
- ORM'in ifade edemediği karmaşık bir sorgu gerektiğinde `text("SELECT ...")` bir kaçış kapısıdır

Tanım: Sayfalama (*pagination*)

Büyük bir sonuç kümesini küçük parçalar hâlinde döndürmedir.

Analoji: Telefon Rehberinin Ciltleri

Analoji

Sayfalamayı bir telefon rehberinin ciltlerine benzetin: binlerce kişiyi tek ciltte basmak yerine ”A–D”, ”E–H” gibi ciltlere (sayfalara) bölersiniz — her seferinde istediğiniz cildi istersiniz, tüm rehberi taşımazsınız.

Kod: Sayfalanmış Sorgu — app/depo.py

```
48    def etkinlik_listele(
49        self, sayfa: int = 1, boyut: int = 20, mekan: str | None = None
50    ) -> list[Etkinlik]:
51        stmt = select(EtkinlikDB).order_by(EtkinlikDB.id)
52        if mekan is not None:
53            stmt = stmt.where(EtkinlikDB.mekan == mekan)
54        stmt = stmt.limit(boyut).offset((sayfa - 1) * boyut)
55        etkinlikler_db = self.db.execute(stmt).scalars().all()
56        return [
57            Etkinlik.model_validate(e, from_attributes=True) for e in etkinlikler_db
58        ]
```

`mekan` parametresi verilmişse bir `where` koşulu eklenir; `limit`/`offset` her zaman uygulanır — filtre ve sayfalama zincirlenebilir.

Kod: echo=True Karşılığı — sorgular.sql

```sql
-- GET /etkinlikler?sayfa=2&boyut=10 karşılığı
SELECT etkinlikler.id, etkinlikler.baslik, etkinlikler.tarih,
     etkinlikler.mekan, etkinlikler.kapasite, etkinlikler.
        bilet_fiyati
FROM etkinlikler
ORDER BY etkinlikler.id
LIMIT 10 OFFSET 10;
```

Konsolda basılan SQL, biçim olarak (takma adlar, sütun sırası) farklı görünebilir ama işlevsel olarak aynı sonucu üretir — bu bir sorun değildir.

BÖLÜM 4

Ünite 4 — Transaction ve Eşzamanlı Rezervasyon

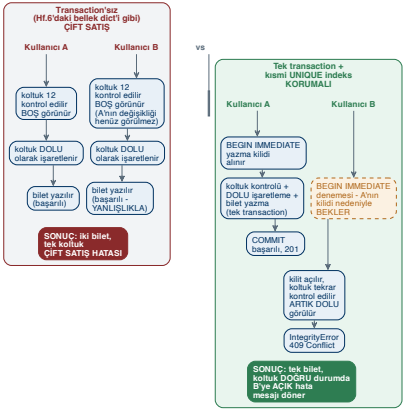
2 • İçerik `commit/rollback`, Tek Transaction, `409`

- Hf.6'nın bellekteki `dict`'inde bu sorun hiç gözlemlenemedi — şimdi veriniz bir dosyada, gerçek bir üretim ortamında birden fazla istek gerçekten eşzamanlı ilerleyebilir
- `db.add(...)` Veritabanına hemen yazılmaz — `db.commit()` çağrılana kadar bir taslak hâlinde bekler; hata olursa `db.rollback()` tüm taslağı geri alır
- "Tek kalem kuralı": kontrol et + dolu yap + bilet yaz — bu üç adım tek bir `Session` ömrü ve tek bir `commit` sınırında yapılır; SQLite'ın `BEGIN IMMEDIATE`'i bu süre boyunca bir yazma kilidi tutar
- İkinci savunma katmanı: kısmi `UNIQUE` indeks — uygulama kontrolü atlanırsa bir `IntegrityError` fırlatır, rota bunu `409`'a çevirir

Tanım: Yarış durumu (*race condition*)

İki eşzamanlı işlemin aynı veriye çakışan biçimde erişip beklenmeyen bir sonuç üretmesidir.

Yarış Durumu: Transaction'sız vs Korumalı



Analoji: Gişe Memuru ve Aynı Anda İki Başvuru

Analoji

”Tek kalem kuralı”nı bir gişe memurunun bir rezervasyonu tek kalemde yazmasına benzetin: ”koltuğu kontrol et, boşsa dolu işaretle, bileti yaz” — bu üç adım ya hep ya hiç olmalı; memur ikinci adımda kalemi bırakıp araya başka bir iş sokarsa defter yarım güncellenmiş, tutarsız kalır.

Analoji

Yarış durumunu aynı koltuğa aynı anda iki gişeden başvuruya benzetin: iki memur aynı anda ”12. koltuk boş mu?” diye bakarsa her ikisi de ”boş” görüp aynı koltuğu iki kişiye satabilir — deftere yazarken birinin diğerini beklemesi (kilit) bu çakışmayı önler.

Kod: Tek-Transaction Rezervasyon (1/2) — Kontrol ve Güncelleme

```
140    def bilet_ekle(self, veri: BiletOlustur) -> Bilet:
152        koltuk_db = self.db.get(KoltukDB, veri.koltuk_id)
153        if koltuk_db is None:
154            raise HTTPException(status_code=404, detail="Koltuk bulunamadı")
155        if koltuk_db.durum == "dolu":
156            raise HTTPException(status_code=409, detail="Bu koltuk zaten satılmış")
157
158        koltuk_db.durum = "dolu"
159        bilet_db = BiletDB(
160            koltuk_id=veri.koltuk_id,
161            kullanici_adi=veri.kullanici_adi,
162            satin_alma_zamani=datetime.now().isoformat(),
163            durum="olusturuldu",
164        )
165        self.db.add(bilet_db)
```

Koltuk `None`/`dolu` kontrolünden sonra `durum = "dolu"` güncellemesi ve yeni `BiletDB` nesnesi aynı `Session` içinde, henüz `commit` edilmeden hazırlanır.

Kod: Tek-Transaction Rezervasyon (2/2) — Commit ve 409

```
167         try:
168             self.db.commit()
169         except IntegrityError:
170             self.db.rollback()
171             raise HTTPException(
172                 status_code=409, detail="Bu koltuk zaten satılmış"
173             ) from None
174         self.db.refresh(bilet_db)
175         return Bilet.model_validate(bilet_db, from_attributes=True)
```

`IntegrityError` yakalanıp `db.rollback()` ile tüm taslak geri alınır, sonra `409`'a çevrilir — kontrol, güncelleme ve yazma aynı `commit` sınırındadır.

Kod: Aynı Adımların Ham SQL Karşılığı — `sorgular.sql`

```sql
-- POST /biletler işleyicisinin TEK transaction'da yaptığı üç adımın
-- (kontrol et + dolu yap + yaz) elle SQL karşılığı
BEGIN IMMEDIATE;

SELECT id, durum FROM koltuklar WHERE id = 1;
-- Uygulama kodu burada durum = 'bos' mu diye kontrol eder.

UPDATE koltuklar SET durum = 'dolu' WHERE id = 1;

INSERT INTO biletler (koltuk_id, kullanici_adi, satin_alma_zamani, durum)
VALUES (1, 'Ayşe Yılmaz', '2026-11-14T10:00:00', 'olusturuldu');

COMMIT;
```

`BEGIN IMMEDIATE` transaction başlar başlamaz bir yazma kilidi alır — aynı anda başka bir transaction'ın yazma niyetiyle başlaması engellenir.

Kod: Transaction Testi — `tests/test_transaction.py`

```python
def test_ardisik_rezervasyon_409(client):
    """Aynı koltuk için ARDIŞIK iki POST /biletler isteği — ikincisi 409 döner."""
    _, koltuk = _etkinlik_ve_koltuk_olustur(client)

    ilk = client.post(
        "/biletler", json={"koltukId": koltuk["id"], "kullaniciAdi": "Ayşe Yılmaz"}
    )
    assert ilk.status_code == 201

    ikinci = client.post(
        "/biletler", json={"koltukId": koltuk["id"], "kullaniciAdi": "Mehmet Demir"}
    )
    assert ikinci.status_code == 409
    assert "satılmış" in ikinci.json()["detail"]

    # Sunucuda TEK bir bilet vardır — ikinci isteğin YAZDIĞI hiçbir kayıt YOK.
    biletler = client.get("/biletler").json()
    assert len(biletler) == 1
```

ilk istek `201` döner, koltuk dolu olur; ikinci istek koltuğun zaten dolu olduğunu transaction içinde görür ve `409` ile reddedilir — veritabanında tek bir bilet kalır.

BÖLÜM 5

Özet

Özet

Özet

- `DeclarativeBase`/`Mapped`/`mapped_column` + `engine`/`Session`/`create_all`: ORM modellerinizi tanımladınız, `bilet.db` dosyasında tablolar oluşturdunuz
- `ForeignKey`/`relationship`: Etkinlik–Koltuk–Bilet ilişkilerini kurdunuz, N+1 sorununu tanıdınız, cascade silme kuralını kesinleştirdiniz
- `select()`/`limit`/`offset`: sayfalanmış sorgular yazdınız, `echo=True` ile üretilen SQL'i okudunuz
- Tek-transaction `commit`/`rollback` + kısmi `UNIQUE` indeks: aynı koltuğa çift satışı gerçek bir transaction'la önlediniz

5 • İzleme Aktarım — Başka Nerede?

- Bugün kurduğunuz `engine/Session/get_db` deseni Hf.10'un `Kullanici` tablosunda, Hf.13'ün arka plan görevlerinde aynen yeniden kullanılacak
- Etkinlik-Koltuk-Bilet ilişki zinciri Hf.10'da `Kullanici`-Bilet ilişkisiyle genişleyecek; `selectinload` deseni Hf.15'in performans ünitesinde derinleşecek
- `echo=True` ile SQL okuma alışkanlığı Hf.15'te "hangi sorgu yavaş" teşhisinde aynen kullanılacak
- Bu haftaki tek-transaction disiplini Hf.13'ün WebSocket bildirimlerinde ve Hf.15'in katmanlı mimarisinde aynı ilkeyle sürecek
- Etkinlik/Bilet Sisteminiz artık kalıcı — sunucu yeniden başlasa, hatta bilgisayar kapanıp açılsa da verileriniz yerinde durur; `kullaniciAdi` alanı ise Hf.10'a kadar düz metin kalmaya devam edecek

Sırada Ne Var?

Bu hafta lab'da

`app/veritabani.py`, `app/orm_modeller.py` ve `app/depo.py`'yi kendi elinizle yazıp `python -m app.tohum` ile örnek veri yükleyecek, sunucuyu yeniden başlatarak verinin yerinde durduğunu görecek, ardışık iki rezervasyon isteğiyle `409`'u kendi gözünüzle gözleyeceksiniz.

- Kendinizi Sınayın her üniteyi kapsıyor: `EtkinlikDB` ile `Etkinlik` neden aynı sınıf değildir, N+1 sorunu nasıl oluşur ve nasıl önlenir, ORM'in ürettiği SQL neden farklı görünebilir, tek-transaction neden gereklidir
- Gelecek hafta: Hafta 9 — Ara Değerlendirme: Hf.1-8'i sentezleyen bir arabölüm, entegre lab ve kapsamlı quiz; yeni konu yok